CS506 Team 18
Project 07 - Plane Peep
Meeting 2026.02.12

Discussion points

Architecture:

- Diagrams
- Sketches
- Draft architecture ideas
  These will help guide your team discussion and future implementation steps.

Demo:

- What did we do in sprint 0
- Have presentations ready

Deliverables:

- `README.md`
- `SKELETON.md`
- `ROLES.md`
- `STYLE.md`
- `LOGO.png`
- .gitignore: .DS_Store, node_modules

Required frameworks:
GitLab
Linux Ubuntu VMs
Docker
IDE: VScode
MySQL
Java/Spring Boot
React/TypeScript
JUnit
Postman
Flight Aware API

Be on the same page (things our grade depends on):

- Make sure we are adding necessary details to issues/MRs
  - Everything needs a **label/assignee/iteration**
  - Add issues/stories to the **NEXT ITERATION**
- Git
  - Never use force (-f, –force), only –force-with-lease)
  - Never commit directly to main

    - Always name your branches
    - Make a commit before walking away from your computer
    - Push commits before walking away from your computer
    - Pair programming: add to commit message
      `Co-authored-by: Name <name@example.com>`
- Who is good at what? (**git especially**, react, java, typescript/nodejs, mysql)
- Rough outline of product
- Be looking out for merge requests to review
- Use discord for any actual project details? Text for informal stuff

Questions for TA:

- Do we only create issues as issues, or can we use issues + tasks
- Is the Scrum Master Report actually daily and in class, or is that supposed to be a part of the WR?
  - Fill out in-class Scrum Master Reports (**SMRs**) each class
    - Answered in piazza
- Do we need merge request approvals
- What are the requirements for specifying AI usage
- Do merge requests need labels
- Does every merge request need a corresponding issue
- **Can we turn off committing to main in gitlab**

This Friday, you will work **in class** with your team on *Architecture Planning*, including the **full-stack design** of your application.

👉 **Come prepared with:**

- Diagrams
- Sketches
- Draft architecture ideas
  These will help guide your team discussion and future implementation steps.

Architecture

Name: Plane Peep

Description: An app that allows users to look up a flight and shows its position on a 2d map. Allows the user to store their favorite flights by creating an account.

Flight API: https://www.flightaware.com

Roles:

- **Database:** Stefan
- **API:** Peter, Stefan
- **Frontend App:** Setrynn, Vincent
- **Backend:** Hunter, Nikolo

Backend API:

- GET /flight/:id/lookup
- GET /account/:accountid/favorite
- POST /account/:accountid/favorite
- POST /account/:accountid/login
- POST /account/create
- GET /flight/:value/search

Diagram:

